certora

Security Assessment Report

Veda

April 2026

Prepared for Veda

certora

Table of Contents

certora

Project Summary

Project Scope

Project Name	Repository Link	Commit Hash	Platform
Veda	https://github.com/Veda-Labs/boring-vault/pull/627/changes	76784f6 (initial commit) 2aeabdb (fix commit)	Solidity

Project Overview

This document describes the security review of **Veda**. The work was undertaken from **April 8th, 2026** to **April 21st, 2026**.

The following contract list is included in our scope:

```
src/base/DecodersAndSanitizers/Protocols/TellerDecoderAndSanitizer.sol
src/base/Roles/BoringQueue/BoringOnChainQueue.sol
src/base/Roles/CrossChain/Bridges/CCIP/ChainlinkCCIPTeller.sol
src/base/Roles/CrossChain/Bridges/LayerZero/LayerZeroTeller.sol
src/base/Roles/CrossChain/Bridges/LayerZero/LayerZeroTellerLib.sol
src/base/Roles/CrossChain/CrossChainTellerLib.sol
src/base/Roles/CrossChain/CrossChainTellerWithGenericBridge.sol
src/base/Roles/AccountantWithYieldStreaming.sol
src/base/Roles/TellerWithMultiAssetSupport.sol
src/base/Roles/TellerWithMultiAssetSupportLib.sol
src/base/Roles/TellerWithYieldStreaming.sol
src/base/IncentivePool.sol
```

The team performed a manual audit of all the files in scope, with a focus on the changes related to the new incentive teller system . During the manual audit, the Certora team discovered bugs in the code, as listed on the following page.

Protocol Overview

This system is a modular vault infrastructure built around a central BoringVault, with teller contracts acting as the main user-facing entry points for deposits, withdrawals, bridging, queueing, and compliance-controlled asset movement. The base teller supports multi-asset deposits and withdrawals, share locks, recipient and transfer restrictions, denylist checks, compliance signatures, optional permit-based flows, incentive reward processing, and principal-history checkpointing for downstream accounting. On top of that, the yield-streaming accountant extends the pricing model by vesting gains over time, updating virtual and realized share prices, tracking time-weighted average supply, and applying losses and fee accounting as vault performance changes. The system also includes an on-chain withdrawal queue for delayed/off-ramped exits, where users escrow shares and later receive assets once requests mature and are solved. Finally, cross-chain teller variants integrate bridge transports such as CCIP and LayerZero, allowing shares to be burned on a source chain and re-minted on a destination chain through authenticated message passing.

Assessment Methodology

Our assessment approach combines design level analysis with a deep review of the implementation to ensure that a protocol is secure, economically sound, and behaves as intended under realistic conditions.

At the design level, we evaluate the architecture, the economic assumptions behind the protocol, and the safety properties that should hold independently of a specific chain or environment. This process includes reviewing internal and cross protocol interactions, state transition flows, trust boundaries, and any mechanism that could be exploited to extract value, deny service, or alter core system behavior. At this stage, a focused threat modelling exercise helps identify key attack surfaces and adversarial capabilities relevant to the system. Design level issues often relate to incentive structures, governance implications, or systemic behavior that emerges under adversarial conditions.

Implementation analysis focuses on the concrete behavior of the code within the execution model of the target chain. This involves reviewing the correctness of logic, access control, state handling, arithmetic behavior, and the nuanced behaviors of the chain environment. Familiar classes of vulnerabilities such as reentrancy conditions, faulty permission checks, precision issues, or unsafe assumptions often surface at this layer. These findings require context aware reasoning that takes into account both the code and the architectural intent.

To support this analysis, the codebase is examined through repeated manual passes and supplemented by automated tools when appropriate. High-risk logic areas receive deeper scrutiny, invariants are validated against both design intent and actual implementation, and potential vulnerability leads are thoroughly investigated. Automated techniques such as static analysis, fuzzing, or symbolic execution may be used to complement manual review and provide additional insight.

Collaboration with the development team plays an important role throughout the audit. This helps confirm expected behaviors, clarify design assumptions, and ensure an accurate understanding of the protocol's intended operation. All findings are documented with clear reasoning, reproducible examples, and actionable recommendations. A follow up review is conducted to validate the applied fixes and verify that no regressions or secondary issues have been introduced.

Threat and Security Overview

At a system level, the protocol is built around BoringVault as the custody and share-issuance layer, with teller, accountant, queue, and bridge modules controlling how users deposit, withdraw, queue exits, earn streamed yield, and move shares across chains. The main security objective is to preserve consistent value accounting and policy enforcement across all of these paths. In particular, share minting and burning should always correspond to valid asset movements or authorized cross-chain state transitions, queued withdrawals should remain fully backed by escrowed shares, and principal / reward accounting should stay aligned with real user exposure. A key invariant is that alternative flows such as permit-based actions, queued exits, yield-streaming withdrawals, or bridged mints must not bypass restrictions that apply to standard deposits and withdrawals.

The main trust boundaries are privileged roles, external dependencies, and cross-chain messaging infrastructure. Authorized roles can pause the system, configure supported assets, set routing and transfer restrictions, manage queue parameters, control bridge peers, and post yield or loss updates, so the system assumes these roles behave honestly and configure the protocol carefully. The design also relies on external pricing inputs, helper contracts, incentive pools, solvers, and bridge-layer authentication / delivery behaving as expected. As a result, the audit treated not only direct theft scenarios as relevant, but also misconfiguration risk, liveness failures, stranded-value scenarios, inconsistent enforcement between modules, and accounting drift caused by valid but unsafe state transitions.

The attack surface considered during the audit covered user deposits and withdrawals, share locks, recipient restrictions, denylist and compliance checks, queue creation / cancellation / solving, reward processing, principal checkpointing, fee and yield realization, admin rescue paths, and cross-chain burn / mint flows. Particular focus was placed on inheritance overrides that may accidentally drop base checks, stale-state griefing in queue operations, low-supply economic edge cases in the yield-streaming accountant, and cross-chain paths that mint shares outside the normal teller validation flow. Overall, the review focused on whether value can be mis-accounted, whether restrictions can be bypassed through an alternate entrypoint, and whether one subsystem can unexpectedly break the safety or liveness guarantees of another.

certora

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	-	-	-
High	-	-	-
Medium	2	2	1
Low	5	5	1
Informational	4	4	-
Total	11	11	2

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
		Likelihood		

certora

Detailed Findings

ID	Title	Severity	Status
M-01	Yield streaming teller bypasses recipient routing restrictions on withdrawal	Medium	Fixed
M-02	When solving a boring queue withdrawal, the checkpointed withdraw value is over/under estimated due to using the wrong rate	Medium	Acknowledged
L-01	Bridged mints bypass normal teller-side deposit checks on the destination chain	Low	Acknowledged
L-02	Bridge compliance check doesn't include destination / bridgeWildCard	Low	Acknowledged
L-03	Whenever reward checkpointing is enabled, P2P transfers and routing should be disabled	Low	Acknowledged
L-04	Unsafe downcast of lastSharePrice can silently corrupt exchangeRate	Low	Fixed
L-05	Users can grief rescueTokens() by temporarily changing queue state	Low	Acknowledged
I-01	Payable functions don't consume ETH	Informational	Acknowledged
I-02	Solmate is being softly deprecated, and no longer actively maintained	Informational	Acknowledged

I-03	depositAndBridge always reverts when share locking is enabled	Informational	Acknowledged
I-04	refundDeposit saves the current rate when updating checkpoints, rather than the rate at time of the deposit	Informational	Acknowledged

Medium Severity Issues

M-01: Yield streaming teller bypasses recipient routing restrictions on withdrawal

Severity: Medium	Impact: Medium	Likelihood: Medium
Files: src/base/Roles/TellerWithYieldStreaming.sol	Status: Fixed	

Description:

In the base TellerWithMultiAssetSupport, both withdraw() and withdrawWithRewards() call _checkRecipient(to) before processing the withdrawal. This is the check that enforces the allowlistedRouterRole restriction and prevents users from routing withdrawals to arbitrary third-party recipients unless they are explicitly allowed to do so. However, TellerWithYieldStreaming overrides these functions and omits _checkRecipient(to) entirely. As a result, users interacting through the yield streaming teller can withdraw to any recipient address even when routing to others is meant to be disabled or restricted to allowlisted routers. This causes the yield streaming implementation to bypass the intended access control / policy restriction present in the base teller.

Recommendations:

Add _checkRecipient(to) to both TellerWithYieldStreaming.withdraw() and TellerWithYieldStreaming.withdrawWithRewards() so the override preserves the same routing restrictions as the base implementation.

Customer Response:

Fixed in 2aeabdb.

Fix Review:

Fix Confirmed.

M-02: When solving a boring queue withdrawal, the checkpointed withdraw value is over/under estimated due to using the wrong rate

Severity: Medium	Impact: Medium	Likelihood: High
Files: BoringOnChainQueue.sol	Status: Acknowledged	

Description:

When a user requests a withdraw through the BoringOnChainQueue:

```
  uint128 amountOfAssets128 = previewAssetsOut(assetOut, amountOfShares,
discount);

        uint40 timeNow = uint40(block.timestamp);
        req = OnChainWithdraw({
            nonce: requestNonce,
            user: user,
            assetOut: assetOut,
            amountOfShares: amountOfShares,
            amountOfAssets: amountOfAssets128,
            creationTime: timeNow,
            secondsToMaturity: secondsToMaturity,
            secondsToDeadline: secondsToDeadline
        });
```

The amount of assets that the user should/will receive is based on the previewAssetsOut function:

```
function previewAssetsOut(address assetOut, uint128 amountOfShares, uint16
discount)
        public
        view
        returns (uint128 amountOfAssets128)
    {
        uint256 price = accountant.getRateInQuoteSafe(ERC20(assetOut));
        price = price.mulDivDown(1e4 - discount, 1e4);
```

```
        uint256 amountOfAssets = uint256(amountOfShares).mulDivDown(price,
ONE_SHARE);
        if (amountOfAssets > type(uint128).max) revert
BoringOnChainQueue__Overflow();
        amountOfAssets128 = uint128(amountOfAssets);
    }
```

The price is based on the current rate, as well as the discount that the user takes advantage of. So the amount of assets calculated here is the exact amount that the user will receive when this withdrawal is solved, i.e. after it matures.

The problem arises when this withdrawal is solved, either self-solved by the user or by an admin, via the solveOnChainWithdraws:

```
function solveOnChainWithdraws(OnChainWithdraw[] calldata requests, bytes
calldata solveData, address solver)
        external
        requiresAuth
    {
        if (isPaused) revert BoringOnChainQueue__Paused();

        ERC20 solveAsset = ERC20(requests[0].assetOut);
        uint256 requiredAssets;
        uint256 totalShares;
        uint256 requestsLength = requests.length;
        for (uint256 i = 0; i < requestsLength; ++i) {
            if (address(solveAsset) != requests[i].assetOut) revert
BoringOnChainQueue__SolveAssetMismatch();
            uint256 maturity = requests[i].creationTime +
requests[i].secondsToMaturity;
            if (block.timestamp < maturity) revert
BoringOnChainQueue__NotMatured();
            uint256 deadline = maturity + requests[i].secondsToDeadline;
            if (block.timestamp > deadline) revert
BoringOnChainQueue__DeadlinePassed();
            requiredAssets += requests[i].amountOfAssets;
            totalShares += requests[i].amountOfShares;
            bytes32 requestId = _dequeueOnChainWithdraw(requests[i]);
            emit OnChainWithdrawSolved(requestId, requests[i].user,
```

```
block.timestamp);
            _onWithdrawSolved(requests[i].user, requests[i].amountOfShares);
        }

        // Transfer shares to solver.
        boringVault.safeTransfer(solver, totalShares);

        // Run callback function if data is provided.
        if (solveData.length > 0) {
            IBoringSolver(solver)
                .boringSolve(
                    msg.sender, address(boringVault), address(solveAsset),
totalShares, requiredAssets, solveData
                );
        }

        for (uint256 i = 0; i < requestsLength; ++i) {
            solveAsset.safeTransferFrom(solver, requests[i].user,
requests[i].amountOfAssets);
        }
    }
```

So, the withdrawal checkpoint will be recorded through _onWithdrawSolved:

```
function _onWithdrawSolved(address user, uint128 amountOfShares) internal virtual
{
        if (principalTeller != address(0)) {

ITellerWithPrincipalTracking(principalTeller).checkpointQueueWithdrawal(user,
amountOfShares);
        }
    }
```

Which will utilize the current rate at this moment:

```
function checkpointQueueWithdrawal(address user, uint256 shares) external
requiresAuth {
        uint256 rate = accountant.getRateSafe();
```

```
        _checkpointPrincipalAtRate(user, shares, false, rate, rate);
    }
```

The current rate might significantly differ from the one which was at the time of making the withdrawal (which actually determined the amount of assets the user will receive). This can significantly over or underestimate the withdrawal amount calculated in _checkpointPrincipalAtRate:

```
} else {
            uint256 baseValue = shares.mulDivUp(baseValueRate, oneShare);
            prevWithdrawals += SafeCast.toUint104(baseValue);
        }
```

With this, the amount recorded at checkpoint will differ from the actual amount the user received (either with or without the discount) affecting the off-chain reward calculations.

Recommendations:

Save the rate used at the time of the withdrawal request and use it to later calculate the checkpoint value.

Customer Response:

Acknowledged.

Low Severity Issues

L-01: Bridged mints bypass normal teller-side deposit checks on the destination chain

Severity: Low	Impact: Low	Likelihood: Low
Files: src/base/Roles/CrossChain/Bridges/CCIP/ChainlinkCCIPTeller.sol	Status: Acknowledged	

Description:

On the destination chain, incoming bridge messages mint shares directly through vault.enter() via the receive path (_ccipReceive() → _completeMessageReceive() → completeMessageReceive()) instead of going through the normal teller deposit flow. Because of that, several checks and side effects that normally happen on a standard deposit are skipped entirely. In particular, bridged mints do not go through the teller's denylist and recipient checks, so destination-side restrictions on who can receive shares are not enforced in the same way as a local deposit. They also bypass deposit compliance verification, meaning any compliance logic expected to run at deposit time is skipped for cross-chain inflows. On top of that, the usual public deposit bookkeeping is not performed, no deposit history entry is recorded, and no normal Deposit event flow is followed.

Recommendations:

Route bridged mints through the same validation and bookkeeping path as standard deposits, or replicate the relevant recipient checks, compliance checks, and deposit tracking logic in the destination chain receive flow before calling vault.enter().

Customer Response:

Acknowledged; Intended behavior and a known issue.

L-02: Bridge compliance check doesn't include destination / bridgeWildCard

Severity: Low	Impact: Low	Likelihood: Medium
Files: CrossChainTellerWithGenericBridge.sol CrossChainTellerLib.sol	Status: Acknowledged	

Description:

Whenever compliance checks are enabled on the bridges and cross-chain tellers, the destination, or bridgeWildcard isn't included as part of the message hash, which enables the users to bridge to any of the destinations allowed on that source.

```
function _verifyBridgeCompliance(
        address sender,
        uint96 shareAmount,
        address to,
        uint256 deadline,
        bytes calldata signature
    ) internal {
        if (complianceSignerRole == type(uint8).max) return;
        bytes32 messageHash = keccak256(abi.encode(address(this), block.chainid,
sender, shareAmount, to, deadline));
        _verifyAndMark(messageHash, deadline, signature);
    }
```

Considering that compliance checks are related to KYC approvals, and that owners of the same address can differ amongst different chains if the address is non-EOA (e.g. smart contract wallets), users could leverage this to bridge to non-approved destinations.

Recommendations:

Include the destination id / bridgeWildcard as part of the message hash and compliance approval.

Customer Response:

Acknowledged.

L-03: Whenever reward checkpointing is enabled, P2P transfers and routing should be disabled

Severity: Low	Impact: Low	Likelihood: Low
Files: TellerWithMultiAssetSupport.sol	Status: Acknowledged	

Description:

Although reward checkpointing is enabled by default, there are instances in which it wouldn't be used, i.e. no rewards/incentives will be calculated/based on its values.

In the instances in which reward checkpointing is active, if P2P transfers aren't disabled, or routing is enabled, this would compromise the integrity of the stored checkpoints.

The idea of the checkpointing system is to always track the current balance of the user within the system, as well as historical changes - so that incentives can be calculated off-chain.

```
    function _checkpointPrincipalAtRate(
        address user,
        uint256 shares,
        bool isDeposit,
        uint256 baseValueRate,
        uint256 currentRate
    ) internal virtual {
        TellerWithMultiAssetSupportLib.checkpointPrincipalAtRate(
            _principalHistory, ONE_SHARE, user, shares, isDeposit, baseValueRate,
currentRate
        );
    }
```

In instances in which P2P transfers are enabled, as well as routing, withdrawals can be wrongly attributed to a different user, or the router-in-question, attributing the withdrawal to a different entity than the one which made the deposit.

Recommendations:

Make sure that P2P transfers (i.e. transfer allowlist) are prohibited, i.e. any transfers outside of mints/burns OR interactions with the boring queue are prohibited.

Additionally, a reward checkpointing on/off switch can be added, similar to the uint8.max mechanism used in the other features.

Customer Response:

Acknowledged.

L-04: Unsafe downcast of lastSharePrice can silently corrupt exchangeRate

Severity: Low	Impact: High	Likelihood: Low
Files: src/base/Roles/AccountantWithYieldStreaming.sol	Status: Fixed	

Description:

AccountantWithYieldStreaming stores vestingState.lastSharePrice as a uint128, but later downcasts it to uint96 in both postLoss() and _collectFees():

```
state.exchangeRate = uint96(vestingState.lastSharePrice);
_calculateFeesOwed(state, uint96(vestingState.lastSharePrice), ...);
```

This is unsafe because Solidity does not revert on explicit narrowing casts from a larger integer type to a smaller one; it silently truncates the high bits. As a result, if vestingState.lastSharePrice ever exceeds type(uint96).max, the protocol does not fail safely and instead stores a wrapped, incorrect exchangeRate. The important part here is that this does not require yield to be posted when supply is already dust. A normal vestYield() can be posted while supply is healthy, then most of the supply can later be withdrawn or bridged out, leaving only a very small residual balance. When _updateExchangeRate() eventually realizes those **previously posted pending gains** against the now tiny currentShares, the share price can spike enough for the uint96 cast to truncate.

A concrete example shows reachability. Assume an 18-decimal vault, so ONE_SHARE = 1e18, RAY = 1e27, and startingExchangeRate = 1e18, giving an initial lastVirtualSharePrice = 1e27. Suppose the vault originally has meaningful supply, yield is posted normally, and later only 1e6 raw share units remain because one user holds almost all of the vault and withdraws nearly everything. If the still-pending vested amount at realization time is just 0.08e18 (0.08 tokens), then during _updateExchangeRate():

- `deltaVirtual = newlyVested * RAY / currentShares = 0.08e18 * 1e27 / 1e6 = 8e37`
- `lastVirtualSharePrice ≈ 8e37`
- `lastSharePrice = lastVirtualSharePrice * ONE_SHARE / RAY ≈ 8e28`

But type(uint96).max ≈ 7.9228e28, so the computed lastSharePrice is already too large for uint96. Once the contract executes uint96(vestingState.lastSharePrice), the value is silently truncated and accountantState.exchangeRate becomes corrupted.

```
function _collectFees() internal {
        AccountantState storage state = accountantState;
        uint256 currentTotalShares = vault.totalSupply();
        uint64 currentTime = uint64(block.timestamp);

        //calculate fees using function inherited from
`AccountantWithRateProviders`
        _calculateFeesOwed(
            state, uint96(vestingState.lastSharePrice), state.exchangeRate,
currentTotalShares, currentTime
        );
```

Also the unsafe downcast happens in postLoss() too ->

```
function postLoss(uint256 lossAmount) external requiresAuth {
    ...
    AccountantState storage state = accountantState;
    state.exchangeRate = uint96(vestingState.lastSharePrice);
    ...
}
```

Recommendations:

Add an explicit bound check before every uint96 cast of vestingState.lastSharePrice, and revert if it exceeds type(uint96).max.

Customer Response:

Fixed in 2aeabdb.

Fix Review:

Fix Confirmed.

L-05: Users can grief rescueTokens() by temporarily changing queue state

Severity: Low	Impact: Low	Likelihood: Low
Files: src/base/Roles/BoringQueue/BoringOnChainQueue.sol	Status: Acknowledged	

Description:

rescueTokens() can be DoSed since it expects the caller to provide an exact, up-to-date snapshot of all active withdrawal requests. A user can sandwich the rescue transaction by first canceling their queued withdrawal, which changes _withdrawRequests and makes the admin's activeRequests array stale, causing rescueTokens() to revert with BoringOnchainQueue__BadInput(). After the rescue fails, the same user can simply submit the withdrawal again and restore their position. This gives the attacker a cheap way to repeatedly block or delay rescue operations by invalidating the admin's snapshot right before execution.

Recommendations:

Avoid relying on a caller supplied live snapshot of queue state inside rescueTokens(). Instead, compute locked shares directly from canonical on-chain request data , else consider acknowledging the issue.

Customer Response:

Acknowledged.

Informational Severity Issues

I-01: Payable functions don't consume ETH

Files:

src/base/Roles/CrossChain/CrossChainTellerWithGenericBridge.sol

Description:

The bridge entry points in CrossChainTellerWithGenericBridge are marked payable, but native ETH is not actually consumed or handled anywhere in the bridge flow. Because of that, if a user mistakenly sends native value along with a bridge transaction, the call can still succeed while the ETH remains stuck in the contract.

Recommendations:

If native ETH is not meant to be consumed in the said flow then the payable keyword can be dropped else consider acknowledging the issue.

Customer Response:

Acknowledged.

I-02: Solmate is being softly deprecated, and no longer actively maintained

Description:

Veda uses the Solmate library for most of their dependencies. Solmate is being softly deprecated, and is no longer actively maintained. Solady has succeeded in this project.

Recommendations:

It's recommended that either Solady or OpenZeppelin dependencies are used for future versions of the system.

Customer Response:

Acknowledged.

I-03: depositAndBridge always reverts when share locking is enabled

Description:

In the CrossChain teller we have the functionality to simultaneously deposit and bridge within the same transaction through the depositAndBridge function. The current way that it's setup, it makes the share locking feature obsolete as it can't be turned on. In cases in which the share locking feature is turned on, the flow will revert.

```
{
        _afterPublicDeposit(
            msg.sender,
            depositParams.depositAsset,
            depositParams.depositAmount,
            sharesBridged,
            shareLockPeriod,
            referralAddress,
            rate
        );

        if (sharesBridged > type(uint96).max) revert
CrossChainTellerWithGenericBridge__UnsafeCastToUint96();
        _bridge(uint96(sharesBridged), to, bridgeWildCard, feeToken, maxFee,
rate);
```

If the shareLockPeriod is > 0, this sets the users share to unlock at block.timestamp + shareLockPeriod;

But at the same time when we reach the _bridge part of the flow:

```
function _bridge(
        uint96 shareAmount,
        address to,
        bytes calldata bridgeWildCard,
        ERC20 feeToken,
        uint256 maxFee,
        uint256 rate
```

```
    ) internal {
        beforeTransfer(msg.sender, address(0), msg.sender);
```

The beforeTransfer check will revert as the shares would be locked.

Customer Response:

Acknowledged.

I-04: refundDeposit saves the current rate when updating checkpoints, rather than the rate at time of the deposit

Description:

refundDeposit updates the checkpoint to account for the refund as a withdrawal and register it in the accounting system. The problem is that the withdraw is calculated with the rate used at the time of the deposit (as it should), while the rate being recorded is the current rate:

```
    emit DepositRefunded(nonce, depositHash, receiver);
        _checkpointPrincipalAtRate(receiver, shareAmount, false,
depositSharePrice, accountant.getRateSafe());
    }
```

This leads to a discrepancy between the actual rate used to calculate the withdrawal and the one being saved in the array:

```
} else {
            uint256 baseValue = shares.mulDivUp(baseValueRate, oneShare);
            prevWithdrawals += SafeCast.toUint104(baseValue);
        }
        principalHistory[user].push(
            PrincipalCheckpoint(uint48(block.timestamp), prevDeposits,
prevWithdrawals, currentRate)
        );
```

Customer Response:

Acknowledged. Intended behavior.

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline. It is helpful for smart contract developers and security researchers during auditing and bug bounties.

Certora also provides services such as auditing, formal verification projects, and incident response.